

Detailed Project on Report on FINANCIAL MARKET ANALYSIS

Submitted By Sagnik Saha

1. Project Overview

This project aims to build a **complete financial data analysis system** that automatically collects stock market data through APIs, stores it in a structured database, and analyses it to generate meaningful insights for investment decisions. In real-world financial markets, raw data is available in large volumes, but without proper processing and analysis, it is difficult to draw useful conclusions.

To solve this problem, the project uses **Python, MySQL, and Exploratory Data Analysis (EDA)** techniques to transform raw stock data into structured information. The system fetches real-time stock data from an external API (Yahoo Finance), stores it in a MySQL database, and performs analysis to understand stock performance, risk, and growth trends. The final outcome helps identify whether a stock is suitable for **long-term investment, short-term trading, or low-risk investing**.

It also delivers an **end-to-end financial data pipeline** that automatically collects market data from external APIs, stores it securely in a **MySQL database**, transforms it into analysis-ready formats, and produces **insightful analytics and reports**, as discussed above. Think of it as a **data highway**: raw market data comes in, gets cleaned and structured, and exits as decision-ready insights. The solution is modular, scalable, and designed with both **technical teams and business stakeholders** in mind.

The Financial Data Pipeline look like this:

API → MySQL → Transformation → Analysis → Final Report

2. Objectives of the Project

The objectives of this project define the purpose, scope, and expected outcomes of the financial data analysis system. Each objective focuses on solving a specific problem related to financial data handling and investment analysis.

1. To automate stock market data collection using APIs

One of the primary objectives of this project is to eliminate manual data collection by using financial APIs. Stock market data changes frequently, and manually collecting this data is time-consuming and error-prone. By using APIs such as Yahoo Finance, the system automatically fetches up-to-date stock information including prices, volume, and timestamps.

This automation ensures:

- Real-time or near real-time data availability

- Accuracy and consistency of financial data
- Reduced human effort and operational cost

Automated data collection also makes the system reliable and suitable for long-term use.

2. To store financial data securely and efficiently using MySQL

Financial data must be stored in a structured and secure manner for future analysis and comparison. This project uses **MySQL**, a relational database management system, to store stock market data in well-defined tables.

Key benefits of using MySQL include:

- Structured storage with proper schema design
- Data integrity and consistency
- Faster querying and filtering using SQL
- Ability to store large historical datasets

Efficient database storage allows the project to scale as data volume grows over time.

3. To clean and transform raw data into analysis-ready formats

Raw data obtained from APIs often contains missing values, inconsistent formats, and unnecessary fields. Another important objective of this project is to clean and preprocess the raw data before analysis.

This includes:

- Removing or handling missing and null values
- Converting date and numeric formats
- Selecting relevant columns for analysis
- Transforming JSON data into CSV and tabular formats

Proper data cleaning ensures that the analysis results are accurate, reliable, and meaningful.

4. To perform detailed Exploratory Data Analysis (EDA)

Exploratory Data Analysis is a key objective of this project. EDA helps in understanding the underlying patterns, trends, and relationships within stock market data before applying advanced models.

Through EDA, the project aims to:

- Identify price trends and growth patterns
- Understand market behaviour and stock stability
- Detect unusual fluctuations or anomalies

- Gain insights using visualizations and statistical measures

EDA acts as the foundation for informed investment decisions.

5. To analyse stock performance using financial metrics

Another major objective is to evaluate stock performance using well-known financial indicators. These metrics provide quantitative measures to assess returns, risk, and consistency.

The project focuses on metrics such as:

- Cumulative return
- Volatility
- Average monthly return
- Percentage of positive closing days
- Volume growth rate

These metrics help compare different stocks and understand their performance over time.

6. To identify low-risk and high-growth stocks

Investors have different risk appetites and investment goals. This project aims to categorize stocks based on their risk and growth potential using analytical results.

- Low-risk stocks are identified through low volatility and stable returns
- High-growth stocks are identified through high cumulative returns and strong volume trends

This objective helps investors choose stocks suitable for **long-term investment, short-term trading, or conservative investment strategies.**

7. To create a scalable base for future AI-driven financial analysis (basically it is a part of Future Scope and Enhancements)

The final objective of this project is to design a system that can be easily extended in the future. The modular architecture allows integration of advanced technologies such as:

- Machine Learning models for price prediction
- AI-based investment recommendations
- Risk-adjusted performance metrics
- Interactive dashboards and real-time analytics

This makes the project future-ready and aligned with modern financial analytics practices.

3. Technology Stack Used

Programming & Database

- **Python** – Used for data ingestion, processing, and analysis
- **MySQL (SQL)** – Used to store and manage structured stock data and primary tool for querying and analysing stock market data.

Analysis & Visualization

- **Pandas** – Data cleaning and manipulation. It is widely used in data science, machine learning, and data engineering owing to its efficiency in handling structured data.
- **NumPy** – Numerical calculations
- **Matplotlib / Seaborn** – Graphical visualization of trends but also creating interactive visualizations to make insights visually compelling and easy to interpret.
- **Jupyter Notebook** – Interactive EDA and analysis. It also has user friendly interface and Graphical visualization with insightful analytics, report.

4. Project Structure and Architecture

The project follows a **layered and modular architecture**, where each layer has a clearly defined responsibility. This design ensures that data flows smoothly from collection to analysis and finally to decision-ready insights. The separation of concerns makes the system **scalable, maintainable, and easy to understand**, even for non-technical stakeholders.

At a high level, the architecture consists of:

1. **Data Layer** – Handles raw and processed data
2. **Database Layer** – Manages structured storage and integrity
3. **Processing Layer** – Automates ingestion, transformation, and retrieval
4. **Analysis Layer** – Performs exploratory and statistical analysis
5. **Reporting & AI Layer** – Converts analytics into business-friendly insights

End-to-End Workflow Explanation

The project follows a modular and systematic workflow:

1. **Data Collection**
Stock market data is fetched from Yahoo Finance using API requests.

2. Data Storage
The raw JSON data is validated and stored in MySQL tables using a predefined schema.
3. Data Retrieval
Stored data is fetched from MySQL using SQL queries for further processing.
4. Data Transformation
Data is converted into CSV format and cleaned for analysis
5. Exploratory Data Analysis
EDA technique is performed to evaluate performance and risk
6. Insight Generation
Final insights are generated in a business-consumable format and results are interpreted to support investment decisions.

```
FINANCIAL_MARKET_ANALYSIS/
├── __pycache__/           # Python cache files
├── .history/              # Editor history (VS Code)
├── data/                  # Raw & processed datasets
│   ├── output.csv         # Processed daily market data
│   └── raw_yahoo_data.json # Raw API response (Yahoo Finance)
├── database/              # Database-related files
│   └── schema.sql         # MySQL table schema
├── notebook/              # Analysis notebooks
│   └── eda.ipynb          # Exploratory Data Analysis (EDA)
├── reports/               # Final outputs & reports
│   ├── analysis_summary.csv # Metrics for AI (volatility, CAGR, trend)
│   └── ai_investment_summary.csv # AI-generated investment summaries
├── src/                   # Core application logic
│   ├── ai_summary.py       # Rule-based + LLM investment summaries
│   ├── api_to_mysql.py     # Fetch market data & store in MySQL
│   ├── fetch_from_mysql.py # Retrieve data from MySQL
│   ├── json_to_csv.py      # Convert raw JSON to CSV
│   └── config.py           # Database & API configuration
├── venv/                  # Python virtual environment
├── README.md              # Project documentation
└── requirements.txt        # Python dependencies
```

Layer-wise Structure Explanation

1. Data Layer (data/)

The **data layer** is responsible for storing all datasets used in the project.

- Stores raw data fetched from the financial API in JSON format
- Stores cleaned and processed data in CSV format
- Acts as the single source of truth for both raw and transformed data

Executive View:

This layer ensures data traceability. Raw data is preserved for auditing, while cleaned data is used for analysis, making the system reliable and transparent.

2. Database Layer (database/)

The **database layer** defines how financial data is structured and stored in MySQL.

- Uses a predefined SQL schema (schema.sql)
- Defines tables, data types, primary keys, and constraints
- Ensures consistency for time-series stock data

Executive View:

A structured database enables fast querying, historical comparisons, and scalable storage as data volume grows over time.

3. Processing & Automation Layer (src/)

This is the **core engine** of the project, where all automation happens.

API to Database (api to mysql.py)

- Fetches stock market data automatically from external APIs
- Validates and cleans incoming data
- Inserts structured records into MySQL

Business Value:

Removes manual effort and ensures consistent, up-to-date financial data.

Configuration Management (config.py)

- Centralizes database credentials and API keys
- Avoids hardcoding sensitive information

Business Value:

Improves security and simplifies environment management.

Data Retrieval (fetch from mysql.py)

- Executes SQL queries to extract stored data
- Prepares datasets for downstream analysis

Business Value:

Ensures repeatable and auditable analytics, which is critical for financial decision-making.

Data Transformation (json to csv.py)

- Converts raw JSON or database outputs into CSV
- Produces analysis-ready datasets

Business Value:

Makes data compatible with Excel (CSV), BI tools for better analysis and visualization, and machine learning pipelines.

4. Analysis Layer (notebook/)

The **analysis layer** focuses on understanding data patterns and performance metrics.

- Uses eda.ipynb for Exploratory Data Analysis
- Analyses price trends, returns, volatility, volume growth, and correlations
- Combines SQL queries with Python-based analytics

Executive View:

This layer answers “*What is happening in the market?*” by turning numbers into understandable trends and risk indicators.

5. Reporting & AI Insight Layer (reports/ + ai_summary.py)

This layer converts analytical outputs into **decision-ready insights**.

AI Investment Summary Engine (ai_summary.py)

- Classifies stocks based on risk (volatility) and growth (CAGR)
- Generates rule-based, explainable investment summaries
- Optionally enhances language using a local LLM (Ollama)
- Includes a safe fallback mechanism if AI enhancement is unavailable
- Outputs investor-ready summaries in CSV format

Executive View:

This layer answers “*What should I do?*” by translating complex metrics into clear investment recommendations.

5. How to run the project

This section explains the step-by-step procedure to set up and execute the Financial Data Analysis project on a local system. The process is designed to be simple and modular so that each stage of the pipeline can be executed and verified independently.

1. Setup Environment: The first step is to create an isolated Python environment to manage project dependencies and avoid conflicts with system-level packages. Install the python packages by *pip install -r requirements.txt*

- *python -m venv venv*
- *.\venv\Scripts\Activate*
- *pip install -r requirements.txt*

2. Create Database: Before ingesting data, the database structure must be created.

- *Run schema.sql in MySQL or VS Code*

3. Ingest Data: Once the database is ready, stock market data is fetched from the external API and stored in MySQL.

- *cd "D:\Financial_Market_Analysis\src"*
- *python api_to_mysql.py*

4. Fetch & Transform Data: After data ingestion, the stored data is retrieved and transformed into analysis-ready formats.

- *cd "D:\Financial_Market_Analysis\src"*
- *python fetch_from_mysql.py*
- *python json_to_csv.py*

5. Analyse: Open the eda.ipynb notebook from the notebook directory

- *Open: notebook/eda.ipynb*

6. Final Report: The final stage converts analytical results into business-friendly insights. This step ensures that complex financial metrics are presented in an easy-to-understand and actionable form.

- *cd "D:\Financial_Market_Analysis\src"*
- *ollama --version (check the ollama version)*
- *ollama list (now check available models)*

You already see a model (e.g. llama3.2, llama3, phi3, mistral)

If no model is listed, pull one:

- *ollama pull llama3*
- *ollama pull phi3*
- *ollama pull mistral*

(any one according to your system capacity)

- *python ai_summary.py*

6. Exploratory Data Analysis (EDA)

This section presents a detailed exploratory analysis of stock market data to extract meaningful insights and graphical visualisation that support informed investment decisions. The analysis is performed using Python-based data processing and visualization techniques, with results stored and summarized through structured CSV reports for business consumption. The examples are given below:

1. df.describe(): Generates **summary statistics** for numerical columns.

Includes:

- Count

- Mean
- Standard Deviation
- Minimum & Maximum values
- Quartiles (25%, 50%, 75%)

```
# Highlights statistical measures including measures of central tendency.
df.describe()
```

[87] ✓ 0.1s

	open_price	close_price	high	low	adjusted_close	volume
count	3514.000000	3514.000000	3514.000000	3514.000000	3514.000000	3.514000e+03
mean	331.794806	331.843680	334.161144	329.243954	329.275998	3.686591e+07
std	174.904622	174.851972	175.536586	174.016651	173.338322	2.687435e+07
min	48.002998	47.833000	48.567001	47.294998	47.833000	1.744200e+06
25%	182.985001	183.132504	184.914997	181.472496	181.900150	1.987948e+07
50%	292.404999	292.694992	293.900009	290.040009	289.302490	3.300540e+07
75%	487.792496	488.167511	491.487511	484.867500	485.189575	4.914235e+07
max	692.190002	694.070007	695.309998	689.320007	694.070007	3.186799e+08

2. **df.head(10)**: Displays the **first 10 rows** of the dataset.

```
# Highlight top 10 rows of the dataset
df.head(10)
```

[89] ✓ 0.0s

	ticker	trade_date	open_price	close_price	high	low	adjusted_close	volume
0	AAPL	2024-01-10T05:00:00.000Z	184.350006	186.190002	186.399994	183.919998	184.448059	46792900
1	AAPL	2024-01-11T05:00:00.000Z	186.539993	185.589996	187.050003	183.619995	183.853683	49128400
2	AAPL	2024-01-12T05:00:00.000Z	186.059998	185.919998	186.740005	185.190002	184.180618	40477800
3	AAPL	2024-01-16T05:00:00.000Z	182.160004	183.630005	184.259995	180.929993	181.912018	65603000
4	AAPL	2024-01-17T05:00:00.000Z	181.270004	182.679993	182.929993	180.300003	180.970901	47317400
5	AAPL	2024-01-18T05:00:00.000Z	186.089996	188.630005	189.139999	185.830002	186.865265	78005800
6	AAPL	2024-01-19T05:00:00.000Z	189.330002	191.559998	191.949997	188.820007	189.767853	68903000
7	AAPL	2024-01-22T05:00:00.000Z	192.300003	193.889999	195.330002	192.259995	192.076050	60133900
8	AAPL	2024-01-23T05:00:00.000Z	195.020004	195.179993	195.750000	193.830002	193.353958	42355600
9	AAPL	2024-01-24T05:00:00.000Z	195.419998	194.500000	196.380005	194.339996	192.680359	53631300

3. **df.tail(10)**: Displays the **last 10 rows** of the dataset.

```
# Highlight last 10 rows of the dataset
df.tail(10)
```

[98] ✓ 0.0s

	ticker	trade_date	open_price	close_price	high	low	adjusted_close	volume
3504	QQQ	2025-12-26T05:00:00.000Z	624.659973	623.890015	625.520020	623.140015	623.890015	28959800
3505	QQQ	2025-12-29T05:00:00.000Z	620.099976	620.869995	622.780029	618.729980	620.869995	32458300
3506	QQQ	2025-12-30T05:00:00.000Z	619.840027	619.429993	622.179993	619.219971	619.429993	31226800
3507	QQQ	2025-12-31T05:00:00.000Z	619.650024	614.309998	619.960022	614.049988	614.309998	40746500
3508	QQQ	2026-01-02T05:00:00.000Z	620.059998	613.119995	622.849976	610.150024	613.119995	61859200
3509	QQQ	2026-01-05T05:00:00.000Z	619.320007	617.989990	620.809998	616.719971	617.989990	46828400
3510	QQQ	2026-01-06T05:00:00.000Z	619.229980	623.419983	624.020020	618.539978	623.419983	43137500
3511	QQQ	2026-01-07T05:00:00.000Z	623.039978	624.020020	627.940002	622.559998	624.020020	44880300
3512	QQQ	2026-01-08T05:00:00.000Z	623.030029	620.469971	623.419983	617.799988	620.469971	50435900
3513	QQQ	2026-01-09T05:00:00.000Z	621.409973	626.650024	627.890015	619.059998	626.650024	49622600

4. `df.isnull().any()`: Checks whether **missing** values exist in each column.

```
# Highlight any null values in the dataset
df.isnull().any()

[91] ✓ 0.0s

...  ticker      False
     trade_date  False
     open_price  False
     close_price False
     high        False
     low         False
     adjusted_close False
     volume      False
     dtype: bool
```

5. `df.dtypes`: Displays the **data type of each column** (int, float, object, datetime etc).

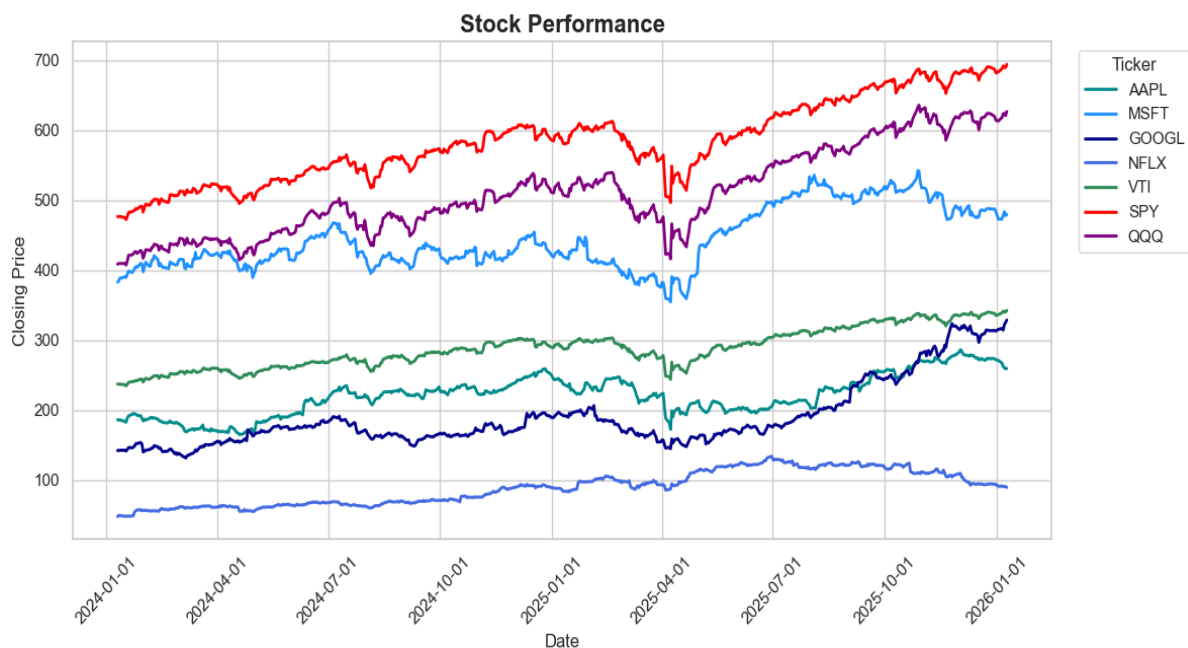
```
# Highlight datatype of each column.
df.dtypes

[92] ✓ 0.0s

...  ticker      object
     trade_date  object
     open_price  float64
     close_price float64
     high        float64
     low         float64
     adjusted_close float64
     volume      int64
     dtype: object
```

6.1 Overall Stock Performance

Overall stock performance is evaluated by analysing the trend of closing prices over time. Time-series plots are used to visualize how stock prices evolve across the selected investment horizon. Here, is the graph given below:



Investment Interpretation:

- **Upward trend** → Suitable for long-term investment and portfolio holding
- **Sideways trend** → Suitable for short-term or swing trading strategies
- **Downward trend** → High-risk investment; requires caution or avoidance

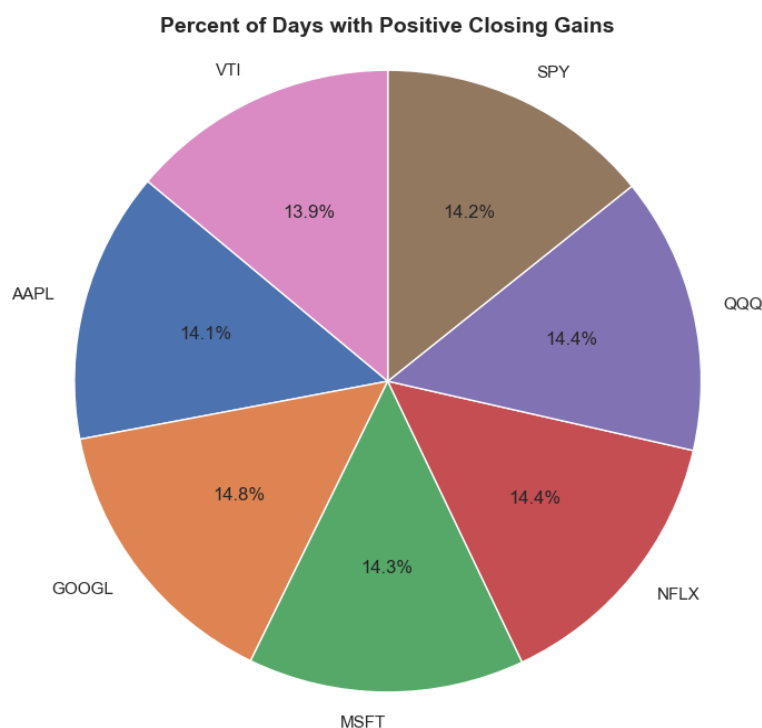
This analysis provides a high-level snapshot of stock behaviour and acts as the first screening step for investors.

6.2 Days with Positive Closing Gains

This metric represents the proportion of trading days where a stock's closing price is higher than its opening price. It provides a clear indicator of the stock's overall bullish trend and market sentiment, offering insights into its performance over the analysed period.

...	ticker	Percentage_Up_Days
0	AAPL	52.589641
1	GOOGL	55.179283
2	MSFT	53.187251
3	NFLX	53.585657
4	QQQ	53.585657
5	SPY	52.988048
6	VTI	51.792829

Create a pie chart to view for how many percent of days prices closed higher than they opened



KEY INSIGHTS

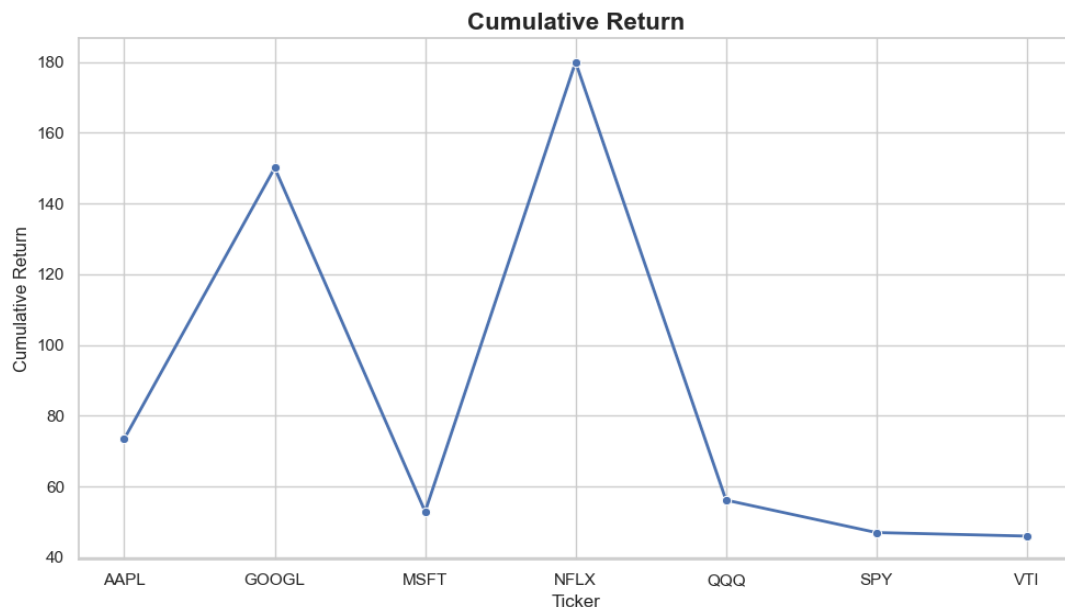
- **Top Performer:** GOOGL (Google), Netflix and Invesco QQQ Trust lead with 55.18%, 53.58% and 53.58% of up days respectively.
- **Neutral Performers:** Apple Inc. and SPDR S&P 500 ETF Trust (SPY) show a rather neutral trend at 52.58% and 52.98% up days respectively.
- **Under Performers:** VTI (Vanguard Total Stock Market Index Fund ETF) have the number of up days at 51.79%.

6.3 Cumulative Return

The Cumulative Return metric reflects the total percentage growth in stock price from its lowest recorded price to its highest. It provides a snapshot of overall return from each stock over the analysed period.

	ticker	Cumulative_Return
0	AAPL	73.448
1	GOOGL	150.053
2	MSFT	52.885
3	NFLX	179.959
4	QQQ	56.128
5	SPY	46.958
6	VTI	45.957

Create a line plot to view cumulative return generated by each stock



KEY INSIGHTS

- **Top Performer:** NFLX (Netflix) has the highest cumulative return at 179.96%.
- **Runner-Up:** GOOGL(Google) follows with an impressive 150.05% return.

- **Moderate Growth:** APPL (Apple Inc) shows a cumulative return of 73.45%, a comparatively lower growth.
- **Underperformer (Relatively):** Vanguard Total Stock Market ETF (VTI) has the lowest cumulative return at 45.96%, which, while significant, is relatively less impressive than others.

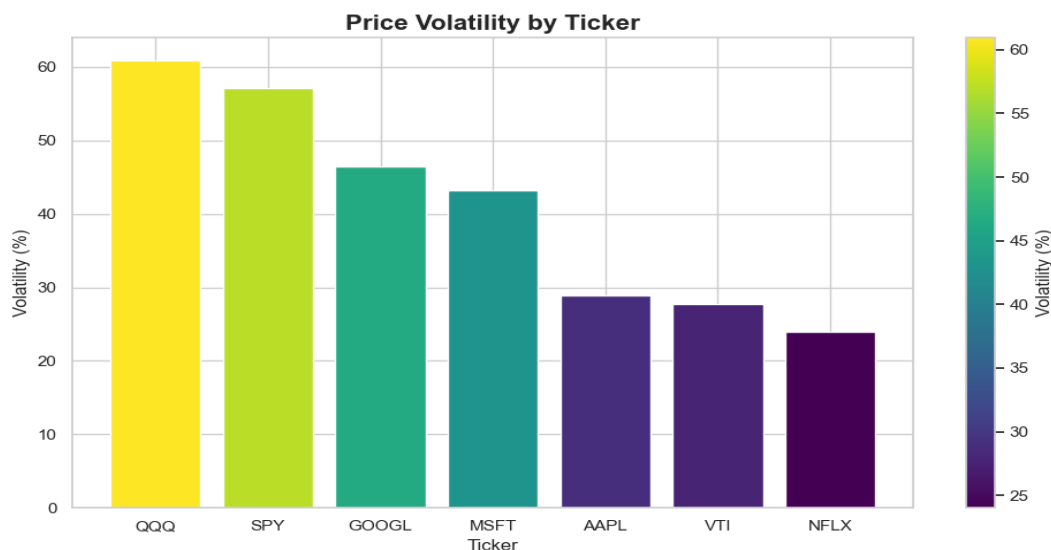
6.4 Volatility Analysis

Volatility shows how much a stock fluctuates over time. Standard Deviation has been used to calculate the deviation of each ticker from its average closing price across the analysed time frame.

- **High volatility:** Higher risk, aggressive price swings, suitable for short-term traders.
- **Low volatility:** Stable returns, suitable for conservative or long-term investors.

..	ticker	Volatility_Percent
0	QQQ	60.922
1	SPY	57.160
2	GOOGL	46.439
3	MSFT	43.163
4	AAPL	28.915
5	VTI	27.749
6	NFLX	24.021

Create a bar graph to indicate volatility of each stock



KEY INSIGHTS

- **Highest Volatility-** The bar chart clearly shows that Invesco QQQ Trust has the highest volatility at 60.92%.
- **Second Highest Volatility-** SPDR S&P 500 ETF Trust (SPY) shows volatility close to NFLX's level at 57.16%.
- **Moderate Volatility-** Microsoft has a considerably lower volatility at 43.16%.

- **Lowest Volatility-** Netflix shows the lowest volatility at 24.06% among the seven stocks.

6.5 Growth Rate of Trading Volume

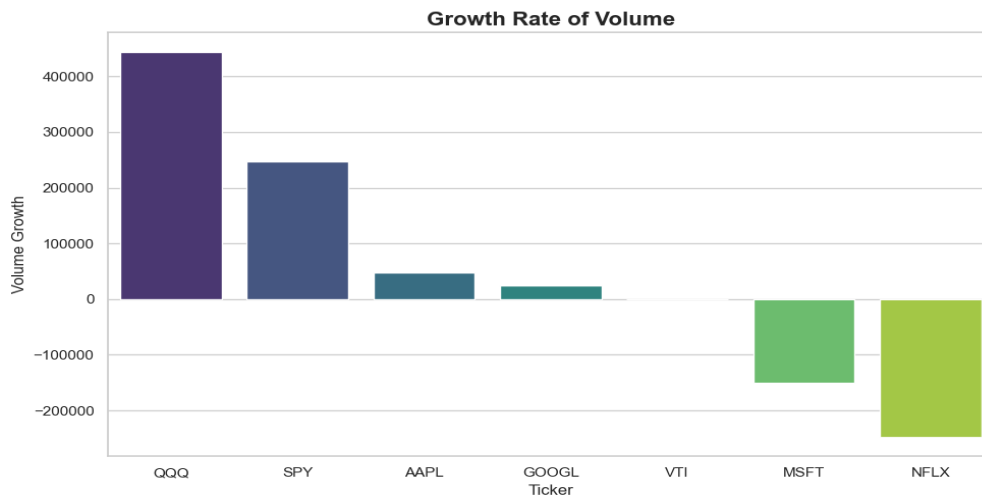
Trading volume growth reflects the level of investor participation and interest in a stock.

Insights:

- **Rising price with rising volume:** Strong bullish trend supported by market participation.
- **Price movement with declining volume:** Weak or unreliable trend.

	ticker	Volume_Trend
0	QQQ	444183.871
1	SPY	247454.839
2	AAPL	47038.710
3	GOOGL	24490.323
4	VTI	88.710
5	MSFT	-149961.290
6	NFLX	-247774.194

Create a bar graph to highlight the volume growth of each stock



KEY INSIGHTS

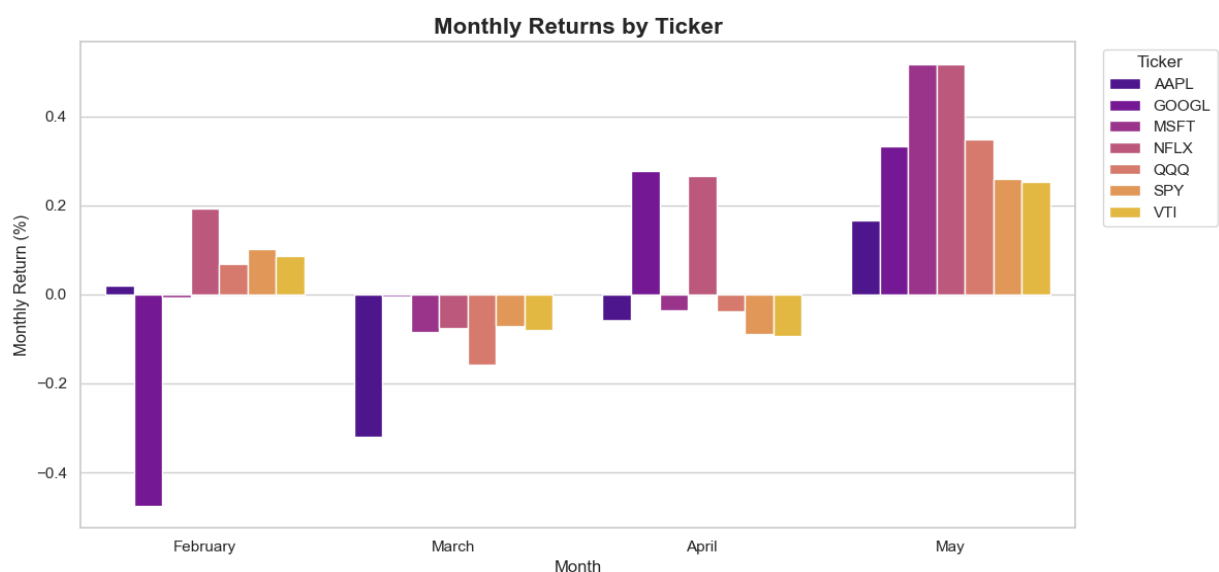
- **Positive Volume Trend:** Invesco QQQ Trust's average trading volume increased by 444,183 shares.
- **Minor Decrease in Volume:** Microsoft showed a small average decrease in trading volume (-149,961 shares).
- **Declining Volume:** Netflix had average trading volume at -247,774 shares respectively.

6.6 Average Monthly Return

Average Monthly Return calculates average closing price of each stock across the analysed months. The same can aid investors in taking prudent & data-driven decisions by understanding when each stock performed better.

	ticker	Month_Name	Monthly_Return_Percent
0	AAPL	February	0.019
1	AAPL	March	-0.320
2	AAPL	April	-0.057
3	AAPL	May	0.167
4	GOOGL	February	-0.475
5	GOOGL	March	-0.005
6	GOOGL	April	0.277
7	GOOGL	May	0.333
8	MSFT	February	-0.006
9	MSFT	March	-0.084
10	MSFT	April	-0.036
11	MSFT	May	0.516
12	NFLX	February	0.193
13	NFLX	March	-0.076
14	NFLX	April	0.266
15	NFLX	May	0.517
16	QQQ	February	0.068
17	QQQ	March	-0.159
18	QQQ	April	-0.037
19	QQQ	May	0.349
20	SPY	February	0.101
21	SPY	March	-0.071
22	SPY	April	-0.089
23	SPY	May	0.260
24	VTI	February	0.087
25	VTI	March	-0.080
26	VTI	April	-0.094
27	VTI	May	0.253

Create a grouped bar graph to highlight monthly return generated by each stock



KEY INSIGHTS

- **Best Performing Month:** The bar chart clearly shows that all 7 stocks had a positive monthly return in May and Netflix has the highest return at 0.517 %.
- **Worst Performing Month:** Clearly the month where returns are negative is February, with lowest return at -0.475 % (Google).

6.7 Correlation Between Price and Volume

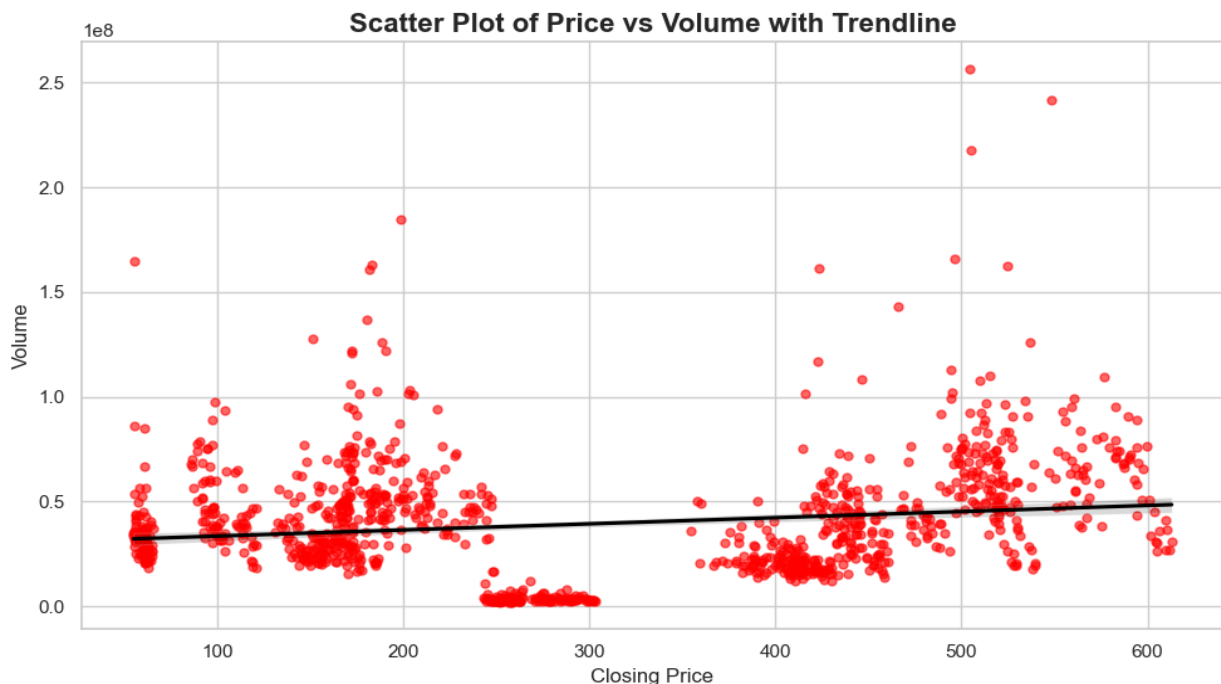
This analysis measures the **statistical relationship between stock price changes and trading volume**. Correlation analysis strengthens confidence in observed market patterns.

Interpretation:

- **Positive correlation:** Price movements are supported by strong volume, indicating reliable trends.
- **Negative or weak correlation:** Price movements may lack market confirmation.

```
... Correlation between Price and Volume: 0.11
```

Create the scatter plot with a trend line to determine existence of correlation between price & volume



KEY INSIGHTS

- **Weak Positive Correlation:** A correlation value of **0.11** indicates a **very weak positive relationship** between the two variables. As one variable increases, the other shows a slight tendency to increase as well, but the effect is minimal.

- **Negligible Relationship Strength:** Since the correlation is close to **0**, the relationship is **not statistically strong**. This suggests that changes in one variable do not meaningfully explain changes in the other.

6.8 Final Stock Performance Analysis Report

This analysis evaluates selected stocks using **three key investment metrics: Annualized Volatility, CAGR (Compound Annual Growth Rate), and Trend Detection** based on moving averages.

1. Volatility Analysis: Volatility measures the degree of price fluctuations over time.

2. Growth Analysis (CAGR): CAGR represents the **annualized growth rate** of each stock over the analysis period.

- **Positive CAGR** signals consistent long-term growth.
- **Negative or low CAGR** reflects underperformance or stagnation.

3. Trend Detection: Trend direction is identified using **50-day and 200-day Simple Moving Averages (SMA)**

- **Uptrend:** Short-term momentum is stronger than long-term trend (bullish signal).
- **Downtrend:** Long-term pressure dominates (bearish signal).
- **Sideways:** No clear directional momentum.

...	ticker	volatility	cagr	trend
0	AAPL	0.4325	0.0620	sideways
1	GOOGL	0.4065	0.1640	sideways
2	MSFT	0.2673	0.1137	sideways
3	NFLX	0.7347	0.7697	sideways
4	QQQ	0.3308	0.1764	sideways
5	SPY	0.2881	0.1651	sideways
6	VTI	0.2907	0.1570	sideways

7. Results and Discussion

Based on the comprehensive Exploratory Data Analysis (EDA) and evaluation of key financial indicators such as cumulative return, volatility, trading volume trends, monthly performance, and growth metrics with positive closing gains, this study provides clear insights into the investment suitability of the selected stocks. The analysis successfully converts raw market data into meaningful, decision-oriented intelligence that supports both long-term and short-term investment strategies.

Top Long-Term Investment Candidates: Long-term investments prioritize consistent growth, positive CAGR, strong cumulative returns, and manageable risk levels.

1. Invesco QQQ Trust (QQQ)

- Highest cumulative return among all stocks
- Strong long-term upward trend
- High growth supported by increasing trading volume
- Suitable for growth-oriented investors with moderate risk appetite

2. SPDR S&P 500 ETF Trust (SPY)

- Stable and diversified market exposure
- Strong cumulative return and positive CAGR
- Balanced volatility profile
- Ideal for conservative to moderate long-term investors

3. Microsoft (MSFT)

- Consistent growth trajectory
- Moderate volatility indicating stability
- Strong business fundamentals reflected in steady performance

Executive Takeaway: These stocks are **well-suited for portfolio holding and long-term wealth creation**, offering a balance of growth and risk control.

Favourable Short-Term Investment: Short-term investments focus on **volatility, frequent price movements, and short-term momentum**, making them suitable for traders and active investors.

1. Netflix (NFLX)

- Lower volatility but sharp short-term price movements
- Suitable for tactical trades and swing strategies

2. Google (GOOGL)

- Highest percentage of positive closing days
- Strong short-term bullish sentiment

- Performs well in momentum-based trading strategies

3. Apple Inc. (AAPL)

- Neutral but stable behaviour
- Suitable for low-risk short-term positioning

Executive Takeaway: These stocks are better aligned with short-term trading strategies, benefiting investors who actively monitor market movements.

Stock Performance by Month (Seasonality Insight): Monthly return analysis highlights **seasonal trends** that can guide timing-based investment decisions.

The monthly return analysis highlights the presence of seasonal patterns in stock performance. The month of **May** stands out as the **best-performing period**, during which all analysed stocks generated positive returns, indicating strong market optimism and investor confidence. **Netflix** recorded the **highest monthly return** during this period, further reinforcing its short-term momentum potential. In contrast, **February** emerged as the **weakest month**, with most stocks showing negative returns and **Google** experiencing the **lowest decline**. This seasonal insight suggests that investment timing plays a critical role in maximizing returns and managing risk.

Executive Takeaway: Seasonality analysis suggests that **entry timing matters**. Investors may benefit from increased exposure during strong months (like May) and defensive strategies during weaker months (like February).

Overall Strategic Conclusion:

- **For long-term investors:** Focus on **QQQ, SPY, and MSFT** for sustained growth and portfolio stability.
- **For short-term traders:** Consider **NFLX, GOOGL, and AAPL** for momentum-driven opportunities.
- **For timing-based strategies:** Leverage **monthly performance trends** to optimize entry and exit decisions.

8. Conclusion

This project clearly demonstrates how raw stock market data can be transformed into **actionable investment insights** using Python, SQL, and Exploratory Data Analysis. By automating data collection and storing it in a structured SQL database, the system ensures accuracy, consistency, and scalability. Python-based analysis enables detailed evaluation of stock performance, risk, and growth trends, while visualizations make complex patterns easy to understand.

A key strength of the project is the use of **AI-generated summaries**, which convert technical analysis into simple, executive-level insights. This allows both technical and non-technical users to quickly assess risk, return, and market behavior before making

investment decisions. Overall, the modular and scalable design makes the system efficient, interpretable, and suitable for real-world financial analysis.

Key highlights:

- Automated data ingestion and SQL-based storage
- Comprehensive performance and risk analysis using Python
- AI-generated summaries for executive understanding
- Scalable and modular analytical pipeline

Outcomes & Impact

- **Automated Financial Data Ingestion:** The system automatically fetches stock market data from external APIs, eliminating manual data collection. This ensures timely, accurate, and consistent financial data availability with minimal human intervention.
- **Centralized SQL-Based Storage:** All financial data is stored in a structured SQL database, enabling secure data management, easy querying, and long-term historical analysis. This centralized approach improves data reliability and scalability.
- **Faster Analysis & Reporting:** Automated pipelines and Python-based analysis significantly reduce the time required to generate insights. Investors and analysts can quickly evaluate stock performance, risk, and trends without repetitive manual processing.
- **Foundation for Dashboards and AI Summaries:** The structured data and analytical outputs create a strong base for building interactive dashboards and AI-generated investment summaries. This allows complex financial insights to be presented in a simple, executive-friendly format.

9. Future Scope and Enhancements

The project has strong potential for further expansion and real-world adoption. Future enhancements can significantly improve predictive power, personalization, and usability.

Key areas of improvement include:

- **Machine Learning-based stock prediction models** to forecast future prices and trends.
- **AI-driven personalized investment recommendations** based on user risk profile and investment horizon.
- **Risk-adjusted metrics** such as Sharpe Ratio, Alpha, and Beta for deeper performance evaluation.
- **Interactive real-time dashboards** using Power BI or Tableau for better decision-making.
- **Cloud deployment** to enable scalability and real-time data processing.
- **Extension to other asset classes** such as mutual funds, ETFs (**Exchange-Traded Fund**), and cryptocurrencies.

With these enhancements, the system can evolve into a **full-fledged intelligent investment analytics platform**, supporting smarter, data-driven investment decisions at scale.

10. References

1. Kaggle Financial & Economic Datasets:
<https://www.kaggle.com/code/aadhya27/stock-price-analysis/notebook>
2. Yahoo Finance: <https://finance.yahoo.com>
3. **Investopedia** – Financial Metrics and Investment Concepts. Reference for definitions of volatility, CAGR, cumulative return, and trading volume analysis.
4. Pandas Documentation: <https://pandas.pydata.org/docs/>
5. Matplotlib: <https://matplotlib.org/stable/index.html>
6. Seaborn: <https://seaborn.pydata.org>
7. Jupyter Notebooks: <https://jupyter.org>
8. SQL Tutorial: <https://www.sqltutorial.org>
9. OpenAI API Documentation: <https://platform.openai.com/docs>
10. Prompt Engineering Guide:
<https://platform.openai.com/docs/guides/promptengineering>
11. **OpenAI**. *ChatGPT*: Used for assistance in generating explanations, structuring analytical content and refining business-oriented interpretations.
URL: <https://chat.openai.com>